

Nome: Eduardo Lucas Lemes Januário CT3037568

Nome: Guilherme Batista CT3037274

Relatório - Trabalho Prático

Introdução

A ordenação é um dos processos fundamentais em ciência da computação e sistemas de informação. Independentemente da linguagem de programação ou do banco de dados utilizado, ordenar dados é uma necessidade prática que aparece em aplicações cotidianas: listagem de nomes, organização de relatórios, classificação de resultados de pesquisas, entre outras.

Embora o objetivo seja simples — reorganizar elementos em determinada ordem — as implementações variam significativamente de acordo com a linguagem e com o contexto. Algumas linguagens utilizam algoritmos híbridos e estáveis, otimizados para cenários reais (como o Python com Timsort), enquanto outras priorizam eficiência geral em grandes volumes de dados (como o Java, que utiliza dual-pivot quicksort para tipos primitivos e Timsort para objetos).

Este relatório investiga como Python e Java implementam algoritmos de ordenação, incluindo o tratamento de textos (strings), critérios de comparação, diferenças entre maiúsculas e minúsculas, e suporte à personalização. Também é analisado o comportamento de ordenação em SQL Server, com foco no funcionamento do ORDER BY, critérios usados por padrão e impacto das configurações de collation. A análise é complementada com exemplos práticos e um quadro comparativo entre as abordagens.

Python

O Python é conhecido por sua simplicidade sintática e por priorizar a produtividade do programador. Uma das razões de sua popularidade é a forma como oferece recursos “prontos para uso” em tarefas comuns. A ordenação de listas é um ótimo exemplo: basta chamar `sort()` ou `sorted()` e o resultado é rápido e confiável. Na minha visão, essa praticidade se justifica porque o algoritmo escolhido, o Timsort, consegue aliar eficiência teórica e desempenho prático, sobretudo em listas parcialmente ordenadas — cenário bastante comum em aplicações reais.

A documentação oficial explica:

“The Timsort algorithm used in Python does multiple sorts efficiently because it can take advantage of any ordering already present in a dataset” (PYTHON SOFTWARE FOUNDATION, 2025).

Isso significa que o algoritmo explora “runs” já ordenados, reduzindo drasticamente o custo em muitos casos. Além disso, o `list.sort()` modifica a lista em memória (in-place), enquanto `sorted()` retorna uma nova lista (PYTHON SOFTWARE FOUNDATION, 2025).

Características

- Estabilidade: mantém a ordem relativa de elementos iguais, essencial em situações como ordenação de registros por múltiplas chaves.

- Complexidade: $O(n)$ $O(n)$ $O(n)$ no melhor caso; $O(n \log n)$ $O(n \log n)$ $O(n \log n)$ no médio e pior caso.
- Espaço extra: pode chegar a $O(n)$ $O(n)$ $O(n)$.
- A escolha do Timsort é coerente com a filosofia do Python, que privilegia robustez em cenários variados. Mesmo que não seja o algoritmo mais rápido em todos os casos, é previsível e estável, características valiosas em ambientes de produção.

Ordenação de textos

Por padrão, Python ordena strings pelo valor Unicode de cada caractere. Isso garante previsibilidade, mas pode gerar resultados não intuitivos em contextos linguísticos (por exemplo, “cão” pode vir depois de “zebra” pelo valor do acento). Nesse ponto, considero essencial o uso de locale ou bibliotecas adicionais, pois apenas assim a ordenação respeitará regras linguísticas.

A documentação aponta que para ordenações dependentes de idioma é necessário recorrer a `locale.strxfrm()` ou `locale.strcoll()` (PYTHON SOFTWARE FOUNDATION, 2025). Dessa forma, o desenvolvedor tem liberdade para escolher entre a ordenação padronizada por Unicode ou uma ordenação culturalmente sensível.

Exemplo em Python

```
import locale

words = ["cachorro", "cão", "café", "Casa", "árvore", "Zebra"]

print(sorted(words)) # Unicode padrão

print(sorted(words, key=str.lower)) # Ignora case

locale.setlocale(locale.LC_COLLATE, 'pt_BR.UTF-8')

print(sorted(words, key=locale.strxfrm)) # Ordenação pt-BR
```

Java

O Java, por sua vez, reflete sua própria filosofia de desempenho e compatibilidade. Trata-se de uma linguagem robusta, voltada a grandes sistemas corporativos, e isso se reflete em sua estratégia de ordenação. Em arrays de tipos primitivos, o uso de dual-pivot quicksort garante excelente desempenho prático, mesmo sem estabilidade. Já em coleções de objetos, a escolha pelo Timsort demonstra a preocupação com consistência e previsibilidade. Na minha opinião, esse modelo híbrido é inteligente: reserva o máximo de desempenho para operações numéricas massivas e oferece estabilidade onde ela é realmente necessária, como em coleções de registros.

Segundo a Oracle:

“The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch” (ORACLE, 2025).

Além disso, em arrays de objetos, o Java aplica uma versão de Timsort semelhante à do Python, garantindo estabilidade (ORACLE, 2025). Isso reforça a ideia de que a linguagem escolhe algoritmos distintos conforme o tipo de dado para otimizar eficiência e corretude.

Características

- Arrays de primitivos (Dual-Pivot Quicksort): rápido, mas não estável. Complexidade média $O(n \log n)$.
- Arrays de objetos e coleções (Timsort): estável, $O(n \log n)$ médio/pior caso.
- Considero que a dualidade é uma escolha estratégica. Em aplicações empresariais, arrays numéricos grandes beneficiam-se da agilidade do quicksort, enquanto coleções de objetos exigem estabilidade para preservar a consistência em ordenações múltiplas.

Ordenação de textos

Em Java, a ordenação de strings com `String.compareTo()` segue valores Unicode, sendo, portanto, case-sensitive e sem tratamento especial para acentos. Isso pode gerar resultados pouco intuitivos em idiomas como o português. Para casos mais sensíveis, a API `Collator` oferece ordenação linguística, o que considero essencial em sistemas que lidam com dados de usuários em diferentes regiões.

A documentação da Oracle sobre `Collator` esclarece:

“The `Collator` class performs locale-sensitive `String` comparison” (ORACLE, 2025).
Isso confirma que o Java oferece suporte robusto para ordenações personalizadas, mas exige configuração explícita por parte do programador.

Exemplo em Java

```
import java.text.Collator;

import java.util.*;

public class OrdenacaoExemplo {

    public static void main(String[] args) {

        List<String> words = Arrays.asList("cachorro", "cão", "café", "Casa",
        "árvore", "Zebra");

        Collections.sort(words);

        System.out.println(words); // Unicode padrão

        Collections.sort(words, String.CASE_INSENSITIVE_ORDER);

        System.out.println(words); // Ignora maiúsculas/minúsculas
```

```

        Collator coll = Collator.getInstance(new Locale("pt", "BR"));

        Collections.sort(words, coll);

        System.out.println(words); // Ordenação pt-BR
    }
}

```

Quadro comparativo

Linguagem	Algoritmo padrão	Complexidade	Estabilidade	Observações
Python	Timsort (list.sort(), sorted())	Médio/pior: $O(n \log n)$ $O(n \setminus \log n)$ $O(n \log n)$, melhor: $O(n)$ $O(n)$ $O(n)$	Estável	Aproveita runs já ordenados
Java (primitivos)	Dual-Pivot Quicksort (Arrays.sort())	Médio/pior: $O(n \log n)$ $O(n \setminus \log n)$ $O(n \log n)$	Não estável	Rápido em prática
Java (objetos)	Timsort (Arrays.sort(Object[]), Collections.sort())	Médio/pior: $O(n \log n)$ $O(n \setminus \log n)$ $O(n \log n)$	Estável	Adequado a grandes coleções

SQL Server (ORDER BY e Collation)

Nos bancos de dados, a ordenação ganha outra dimensão: além do algoritmo interno, entram em cena regras linguísticas e culturais. O SQL Server é um exemplo claro, pois sua ordenação é determinada pela *collation*, que define se a comparação é sensível a maiúsculas, minúsculas e acentos. Na minha visão, isso torna o banco de dados altamente configurável, mas também exige atenção do desenvolvedor: escolher a collation errada pode levar a resultados incorretos ou inesperados em consultas.

Segundo a Microsoft:

“Collations determine the rules that sort and compare data” (MICROSOFT, 2025).

Ou seja, a collation é responsável por estabelecer as regras de ordenação e comparação de textos em nível de servidor, banco, tabela ou coluna. Isso implica que a mesma consulta pode retornar resultados diferentes dependendo da configuração.

Critério padrão

Cada banco, coluna ou consulta pode definir uma collation específica.

As principais opções incluem:

- `_CI_`: case-insensitive.
- `_CS_`: case-sensitive.
- `_AI_`: accent-insensitive.
- `_AS_`: accent-sensitive.

Na prática, significa que “Ana” e “ana” podem ser considerados iguais em uma collation case-insensitive, mas distintos em uma case-sensitive.

Diferença números x textos

- `ORDER BY idade` → ordenação numérica é ordenada de forma direta e objetiva.
- `ORDER BY nome` → Em cenários multi-idíomas, considero essencial especificar a collation em cada consulta para garantir previsibilidade, sendo a consulta realizada com base na collation.

Exemplo em SQL Server

```
CREATE TABLE Pessoas (  
    id INT PRIMARY KEY,  
    nome NVARCHAR(100) COLLATE Latin1_General_CI_AS,  
    idade INT  
);  
  
INSERT INTO Pessoas (id, nome, idade) VALUES  
    (1, 'Ana', 30),  
    (2, 'Álvaro', 25),  
    (3, 'alberto', 40),  
    (4, 'andre', 22),  
    (5, 'Àgata', 35),  
    -- Ordenação numérica  
  
SELECT nome, idade FROM Pessoas ORDER BY idade;
```

-- Ordenação padrão (accent-sensitive)

```
SELECT nome FROM Pessoas ORDER BY nome;
```

-- Ignorando acentos

```
SELECT nome FROM Pessoas ORDER BY nome COLLATE Latin1_General_CI_AI;
```

Conclusão

Ao comparar Python, Java e SQL Server, fica claro que a ordenação é um tema que ultrapassa os limites da teoria algorítmica. Cada tecnologia faz escolhas diferentes, refletindo sua filosofia e público-alvo.

O Python prioriza a robustez e a estabilidade com o Timsort, privilegiando clareza e previsibilidade em detrimento da máxima velocidade em todos os casos. O Java adota uma postura híbrida, equilibrando rapidez em arrays numéricos (com quicksort) e confiabilidade em coleções de objetos (com Timsort). Já o SQL Server mostra que no mundo dos bancos de dados a ordenação vai além dos algoritmos, dependendo fortemente de collations que incorporam aspectos linguísticos e culturais.

Pessoalmente, entendo que não há um “melhor” sistema de ordenação universal. O importante é reconhecer o contexto: listas pequenas podem se beneficiar de algoritmos simples, arrays numéricos grandes de quicksort, coleções heterogêneas de Timsort, e bancos de dados de collation configurada corretamente. O valor desse estudo é justamente perceber que ordenar dados é tanto uma questão de eficiência algorítmica quanto de sensibilidade cultural e linguística.

Referências

- PYTHON SOFTWARE FOUNDATION. *Sorting HOWTO*. Disponível em: <https://docs.python.org/3/howto/sorting.html>. Acesso em: 13 set. 2025.
- PYTHON SOFTWARE FOUNDATION. *Built-in Types*. Disponível em: <https://docs.python.org/3/library/stdtypes.html#list.sort>. Acesso em: 13 set. 2025.
- ORACLE. *Class Arrays (Java Platform SE 21)*. Disponível em: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Arrays.html>. Acesso em: 16 set. 2025.
- ORACLE. *Class Collections (Java Platform SE 21)*. Disponível em: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/Collections.html>. Acesso em: 16 set. 2025.
- MICROSOFT. *Collations (Transact-SQL)*. Disponível em: <https://learn.microsoft.com/en-us/sql/t-sql/statements/collations>. Acesso em: 18 set. 2025.